Ders 8 Genişletilmiş Notu: Fonksiyonlar, Prototip, Değerle ve Referansla Çağırma, Bubble Sort ve İkili Arama

Ders 8 Genişletilmiş Notu: Fonksiyonlar, Prototip, Değerle ve Referansla Çağırma, Bubble Sort ve İkili Arama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bir Problemle Başlamak: Kod Tekrarı
- 2. Modüller, Kütüphane Fonksiyonları ve Kullanıcı Tanımlı Fonksiyonlar
- 3. Fonksiyon Çağrısı
- 4. Fonksiyon Tanımı ve Dört Durum
- 5. Düşük Bağlaşım, Yüksek Uyum
- 6. Matematik Kütüphanesi
- 7. Yerel Değişkenler ve Böl ve Fethet
- 8. Fonksiyonun Genel Biçimi ve return
- 9. Fonksiyon Prototipi
- 10. Değerle Çağırma ve Referansla Çağırma
- 11. Dizilerin Özellikleri
- 12. Karakter Dizileri
- 13. Dizilerin Fonksiyonlara Aktarılması
- 14. Bubble Sort: Kâğıt Üzerinde
- 15. Bubble Sort ve Dizi Yazdırma Fonksiyonlarını Kodlamak
- 16. swap Deneyi: Değer ile Referansın Farkı
- 17. Sıralı Arama
- 18. İkili Arama
- 19. Sayaçlarla Karşılaştırma ve Dersin Tam Programı

1. Bir Problemle Başlamak: Kod Tekrarı

Hoca konuya girmeden önce bir kod yazdı. Amacı, karşılaşacağımız bir problemi göstermek ve bunu nasıl çözebileceğimizi sormaktı.

Önce dizi oluşturmayı hatırlattı. C'de 10 elemanlı bir dizinin elemanlarını klavyeden girebildiğimiz gibi, tanımlarken ilk değer de verebiliriz (initialization). 10 elemanın tamamını yazarsak dizi yazdığımız değerleri alır. Daha az eleman yazarsak, örneğin 5 eleman, ilk 5 eleman verdiğimiz değerleri alır, diğerleri otomatik olarak 0 yapılır.

Hoca 10 elemanlı, sıralı bir dizi tanımladı. Kodun her yerinde 10 yazmamak için de boyutu bir sembolle verdi:

```
#define MAX 10
```

Sonra dizinin elemanlarını bir `for` döngüsüyle ekrana yazdırdı. `printf` içindeki `%3d`, sayının yazılması için 3 karakterlik alan açar. Buraya kadarki kısım daha önceki derslerde görülmüştü.

Ardından şöyle bir varsayım yaptı: Bir sonraki satırda dizi üzerinde bir işlem yapıyoruz ve içeriği değiştiriyoruz. Örneğin 0. elemanı 1 yapıyoruz, 0. ve 1. elemanı toplayıp sonucu 2. elemana atıyoruz. Dizinin sıralı kalmasına dikkat etti. 1. eleman 5 olduğu için toplam $1+5=6$ olur ve dizi yine sıralıdır.

Dizinin son hâlini görmek istediğimizde ne yaparız? Yazdırma kodunu kopyalar, yapıştırırız. Araya başka bir iş girip diziyi yine görmek istesek aynı kodu bir daha kopyalarız. Hoca buna **kod kokusu (code smell)**, yani kod kusuru dedi. Üç satırlık bir kodda bu makul görünebilir. Ama aynı iş 30, 40 ya da 500 satırlık bir kodla yapılıyorsa ve birden fazla kez gerekiyorsa kopyalamak makul olmaktan çıkar.

Sorunun asıl yüzü değişiklik sırasında görülür. Yazdırma kodundaki `%3d`'nin fazla olduğuna karar verip `%2d` yapmak istersek bu değişikliği her kopyada ayrı ayrı yapmamız gerekir. Örnek basit olduğu için önemsiz görünebilir. Ama 10 satırlık bir kod yazıp kopyaladıktan sonra yanlış yazdığınızı fark ederseniz, düzeltmeyi bütün kopyalara yansıtmanız gerekir. Oysa kodu bir kez yazıp yalnız ilgili yerden değiştirmek ciddi bir kolaylık sağlar. `#define MAX 10` satırını değiştirince bütün kodun bundan etkilenmesi gibi, fonksiyonlarda da bir değişikliği tek yerde yaparız.

2. Modüller, Kütüphane Fonksiyonları ve Kullanıcı Tanımlı Fonksiyonlar

Bu sorunun üstesinden gelmek için birçok programlama dilinin sağladığı kolaylık, programı parçalara bölmektir. Yapısal programlamanın en büyük faydalarından biri modülerliktir, yani programın modüllere ayrılmasıdır. Amaç, kod tekrarını engelleyecek biçimde daha küçük parçalarla program yazmaktır. Bu parçalar sayesinde program daha kolay yönetilir ve bakımı daha kolay yapılır.

C'de modüllere **fonksiyon** denir. Fonksiyonlar iki gruptan gelir:

- **Kütüphane fonksiyonları (library functions):** `printf`, `scanf`, `floor` gibi fonksiyonlar ve matematik fonksiyonları.
- **Kullanıcı tanımlı fonksiyonlar (user-defined functions):** Bizim yazdığımız fonksiyonlar. Ekrana yazdır, karesini al, diziyi sırala gibi.

Aslında fonksiyon çağrısını zaten kullanıyorduk. `printf` çağırdığımızda onun kodu programımızın içine yapıştırılmaz. Derleme sırasında da yeniden derlenmez, çünkü zaten derlenmiştir. Derlemeden sonra bağlayıcı (linker) bu derlenmiş kodu getirir ve program çalıştırılabilir hâle gelir. Kullanıcı tanımlı fonksiyonlarla biz de aynı rahatlıktan yararlanacağız.

Hoca dersin planını da söyledi: Önce slayttan fonksiyonları anlatacak, sonra uygulamayı arama ve sıralama algoritmaları üzerinden yapacak. Fonksiyon kullanımında iki önemli kavram vardır: fonksiyonun **tanımı (definition)** ve fonksiyonun **çağrısı (call)**.

3. Fonksiyon Çağrısı

Bir `printf` çağırmak bir fonksiyon çağrısıdır. Burada kullanılan terim **invoke**'tur. Sözlükte yalvarmak, dua etmek gibi anlamlar görülür, ama bilgisayar bilimlerinde çağırmak anlamında kullanılır: function invoke, invoking of function. Hoca benzer iki kelimeyi de andı:

- **Invoke:** Çağırmak (call).
- **Revoke:** Bir şeyi iptal etmek, yürürlükten kaldırmak.
- **Evoke:** Uyandırmak, hatırlatmak, dürtmek, tetiklemek. Mikroişlemcili sistemlerde kesme (interrupt) mekanizmasıyla ilgili olarak kullanılır.

Bir fonksiyonu çağırırken adını yazarız ve varsa argümanlarını veririz. `printf` çağrısında `printf` fonksiyonun adıdır, parantez içinde tırnakla yazdığımız ve virgülle eklediğimiz şeyler argümanlarıdır. Hocaya göre `scanf` en az iki argüman almak zorundadır, `printf` için ise bir argüman yeterlidir. Bu, o fonksiyonların tanımından gelen bir durumdur.

> **Ders sonrası not:** Hoca `scanf`'in kesinlikle en az iki argüman aldığını söyledi. Bir değer okumak için bu doğrudur, çünkü biçim metninin yanında bir de adres gerekir. Dil kuralı olarak ise büyük olasılıkla yalnız biçim metni zorunludur: `scanf`'in prototipinde biçim parametresinden sonra değişken sayıda argüman gelir.

Argüman ile **parametre** ayrı kavramlardır. Çağrıda gönderdiğimiz verilere argüman, fonksiyonun tanımlandığı yerde bu verileri alan değişkenlere parametre denir.

Soru: Argüman ile parametre aynı şey mi? **Cevap:** Aynı şey değil. Argüman fonksiyon çağrısında gönderilen verileri, parametre fonksiyonun tanımında alınan verileri ifade eder.

Fonksiyonlar bazı işlemleri ve düzenlemeleri yapar, örneğin diziyi sıralar ya da ekrana yazar. Elde edilen sonucu da bize geri döndürebilirler.

3.1 Patron ve İşçi Benzetmesi

Hocanın benzetmesi şudur: Bir patron işçisine bir işi tamamlamasını söyler. İşçi gerekli bilgileri toplar, işi yapar ve sonucu geri verir. Patron işin nasıl yapıldığının ayrıntılarını bilmez. Buna **bilginin gizlenmesi (information hiding)** denir.

Biz de `scanf` ve `printf`'in nasıl çalıştığını şu an bilmiyoruz ve bilmemize gerek de yok. Alt seviye programlama yaparsak bunu ileride öğrenmemiz gerekebilir. Fonksiyonun verimli çalışması ve iyi bir algoritma içermesi onu kodlayan kişinin işidir. Çağıran kişiyi bu ayrıntı pek ilgilendirmez. Programlar da birçok fonksiyon çağrısıyla yönetilir.

Bir C programı genel olarak şöyle bölünür:

1. `main` fonksiyonu, `worker` diye anılan alt fonksiyonları (worker 1, worker 2, worker 3) çağırır.
2. Veriler argüman olarak gider, alt fonksiyonlar bunları parametre olarak alır ve işlemlerini yapar.
3. Alt fonksiyon geriye bir sonuç döndürür (return) ya da döndürmez.
4. Bir worker kendi içinde başka fonksiyonları çağırabilir.

Fonksiyonları birçok kez kullanabiliriz. Bu, büyük programlarda yazılan metnin karmaşıklığını azaltır ve kodu daha düzenli hâle getirir.

4. Fonksiyon Tanımı ve Dört Durum

Fonksiyonun tanımında iki bağımsız soru vardır: Fonksiyon parametre alıyor mu, değer döndürüyor mu? Biri fonksiyona gelen verilerle, diğeri fonksiyondan çıkan değerle ilgilidir. Bağımsız oldukları için $2 \times 2 = 4$ durum vardır:

- Parametre alır, değer döndürür.
- Parametre almaz, değer döndürür.
- Parametre alır, değer döndürmez.
- Parametre almaz, değer döndürmez.

Döndürülen değerden kastımız, `int`, karakter, `float` gibi temel tipler ya da kendi tanımlayacağımız tiplerdir. Örneğin kendiniz `book` diye bir tip üretip o tipte bir değer döndürebilirsiniz. Tamsayı işaretçisi de (`int` pointer) döndürülebilir. Bunları ileride göreceğiz.

4.1 Değer Döndürmeyen, Parametre Almayan Fonksiyon

Dönüş değeri yoksa başa `void` yazılır, sonra fonksiyonun adı, ardından parantez gelir. Bir blok açılır ve kodlar içine yazılır. Çağırırken yalnız adı ve parantez yazılır:

```
void hello(){
    /* yapılacak işler */
}

/* çağrı */
hello();
```

`void` İngilizcede "boş" demektir. Hoca bunun için "null gibi" de diyebileceğimizi söyledi. Dersteki örnek fonksiyon, hocanın kendi deyimiyle anlamsız ama öğretici bir fonksiyondu. Tek yaptığı alt satıra geçmekti:

```c
void altsatiragec(){
    printf("\n");
}
```

Bir satırlık bir işi fonksiyona çevirmenin gerçekte anlamı yoktur, ama örnek için yazıldı ve çıktının değişmediği görüldü.

4.2 Çağrı Sırasında Ne Olur?

Hoca, programdaki satır numaralarıyla akışı anlattı. Çağrı 12. satırda, fonksiyon ise 18. satırdan başlıyordu.

1. Program 12. satırdaki çağrıya gelir. İşlemcideki program sayacı (program counter) 18. satırdaki kodları çalıştırmaya başlar.
2. Daha sonra çağrının yapıldığı yere dönmesi gerektiği bir yerde saklanır.
3. 18. satırdan itibaren kapatan süslü paranteze (20. satır) kadar çalışılır.
4. İş bitince çağrının yapıldığı yere dönülür ve program 13. satırdan devam eder.

Soru: Eskiden Dev-C++ fonksiyon `main`'in üstünde tanımlanmazsa hata veriyordu. Şimdi neden vermedi? **Cevap:** Bu prototiple ve programın derlenmesiyle ilgili ayrı bir konu. C'nin farklı sürümlerinde o hatayı alabilirsiniz. Birazdan bunu önlemek için ne yapılacağı konuşulacak.

4.3 Değer Döndüren Fonksiyon

Fonksiyon bir değer döndürecekse başa döndüreceği tip yazılır ve `return` ile değer döndürülür:

```c
int get(){
    /* ... */
    return x;
}

/* çağrı */
t = get();
```

Fonksiyon parametre almadığı için çağrıda argüman gönderilmez. Gönderirsek hata olur. Dönüş değeri `int` olduğu için `t` diye bir tamsayı değişken tanımlarız. Önce fonksiyonun işlemi yapılır, `return` edilen değer sonra `t`'ye atanır.

C'de `return` ile **tek bir değer** döndürülür. İki ya da üç değer döndürmek mümkün değildir, başka dillerde bu mümkün olabilir. Tek bir değişken ya da tek bir dizi elemanı döndürülebilir, örneğin `a[i]` veya `x`. Hatta `x + y` gibi bir ifade de yazılabilir: Önce hesaplanır, sonucu döndürülür. Ama bir dizinin tamamı döndürülemez.

4.4 Parametre Alan Fonksiyon

Fonksiyon parametre alacaksa, parantez içinde bir değişken tanımlar gibi parametre tanımlanır. Bir sayının küpünü alan fonksiyon şöyledir:

```c
int cube(int n){
    return n * n * n;
}

/* çağrı */
t = cube(3);
```

`return n * n * n;` önce çarpmaları yap, sonra sonucu tamsayı olarak döndür demektir. Bu çağrı `main`'de olabileceği gibi başka bir fonksiyonda da olabilir. Hoca, fonksiyonun içeriğine gerçekleştirim (implementation) dendiğini de hatırlattı.

İki ya da üç parametre için araya virgül konur. `add` fonksiyonu 2 ve 7 ile çağrıldığında $2+7=9$ sonucu `t`'ye atanır (ara adım eklendi):

```c
int add(int a, int b){
    return a + b;
}
```

```
/* çağrı */
t = add(2, 7);
```

Dönen değeri bir değişkene almak zorunda değiliz. `cube(4);` yazarsak değer döner ama hiçbir yerde kullanılmaz, anlamsız bir biçimde boşta kalır. `printf("%d", cube(7));` yazarsak dönen değer `%d` ile ilişkilendirilir ve ekrana yazılır. Üç kullanım da C'de geçerlidir. Hocaya göre bazı dillerde hepsi geçerli olmayabilir, çünkü C kontrolü daima programcıya bırakır.

Her parametre ayrı ayrı tanımlanmak zorundadır. `int add(int a, b)` yazılamaz.

Soru: Burada `public` diye bir şey var mı? **Cevap:** C'de `public` diye bir şey yok.

Slaytta `add`'in sonucunun `value2` gibi bir değişkene kopyalandığı gösteriliyordu. Yukarıdaki kısa gösterimler ders sonrasında küçük bir programla `gcc -Wall -std=c89 -pedantic` altında uyarısız derlendi; `cube(3)` için 27, `cube(7)` için 343 ve `add(2, 7)` için 9 elde edildi.

5. Düşük Bağlaşım, Yüksek Uyum

Hoca bu slaydı, başka bir hocayla verdikleri Nesneye Yönelik Programlama dersinden aldığını söyledi. Fonksiyon tasarımında iki kavram vardır.

Bağlaşım (coupling), yapışıklık, bağlılık demektir ve düşük olmalıdır. Hoca bunu şu örnekle açıkladı: Fonksiyon birden fazla işi değil, tek bir işi yerine getirmelidir. Diziyi sıralamak için `sirala` diye bir fonksiyon yazdıysanız, orada yalnız sıralamayla ilgili işlemler yapılmalıdır. Sıraladıktan sonra "hadi yazdırayım, sonucu göreyim" derseniz bu istenmez. Üç elemanı sıralayıp yazdırmak bir şey değildir, ama 3 milyon elemanı sıralayıp bir de yazdırmak ağır bir yüktür. Yazdırmak isteyen önce `sirala`'yı, sonra `yazdir`'ı ayrı ayrı çağırır. Bu daha makuldür ve modülerliği sağlar. Düşük bağlaşım sayesinde yapılan bir değişiklik diğer tarafları daha az etkiler.

Uyum (cohesion), parçaların sorumluluklarının birbiriyle uyumlu olma oranıdır. Fonksiyonun adı ile yaptığı iş uyumlu olmalıdır. Hoca, "coherent" kelimesinin tutarlı anlamına geldiğini de hatırlattı, kohezyon da buna benzer. `sirala` adlı bir fonksiyonda sıralamanın yanında alakasız bir matris çarpımı yapılması akla mantığa aykırıdır. Uyum yüksek (high cohesion), yani tutarlı olmalıdır.

Faydaları şunlardır: anlaşılma kolaylığı artar, yeniden kullanılabilirlik artar, bakım kolaylaşır, değişikliklerden etkilenme olasılığı düşer. Düşük bağlaşım genellikle yüksek uyumu beraberinde getirir, yüksek bağlaşım da düşük uyumu.

Soru: Sıralamadan sonra yazdırmayı çağırmak linkleme mi oluyor? **Cevap:** Hayır. Bunlar ayrı fonksiyonlardır, istersem çağırırım, istersem çağırmam. Mesele, bir fonksiyonun içine bir sürü alakasız kod yazmamaktır.

> **Ders sonrası not:** Hoca "tek bir iş yapmak" ilkesini bağlaşım başlığı altında anlattı. Terimlerin yaygın anlamı büyük olasılıkla şöyledir: Bir fonksiyonun tek ve tutarlı bir iş yapması uyumla (cohesion), parçaların birbirine ne kadar bağımlı olduğu ise bağlaşımla (coupling) ilgilidir. Hocanın verdiği tavsiye bu ayrımdan bağımsız olarak geçerlidir.

6. Matematik Kütüphanesi

Matematik kütüphanesindeki fonksiyonları kullanmak için önce kütüphane `include` edilir, sonra fonksiyonun adı ve argümanları yazılır. `sqrt` çağrısında argüman olarak 900 verilirse fonksiyon bir değer döndürür. Bu değer `printf`'teki biçim belirteciyle ilişkilendirilir (hoca sonucu söylemedi; $\sqrt{900} = 30$, ara adım eklendi).

C'de bütün matematik fonksiyonlarının dönüş tipi `double`'dır. Argüman üç biçimde olabilir:

- **Sabit sayı:** `sqrt(900)`.
- **Değişken:** `sqrt(x)`. `x`'in değeri neyse oraya kopyalanır.
- **İfade:** `sqrt(x + 2)`. Önce `x` ile 2 toplanır, sonra değer gönderilir.

Slaytta karekök, e^x ve sinüs gibi fonksiyonların çalışma örnekleri vardı. Hoca bunların ezberlenmesine gerek olmadığını, bilinmesinin yeterli olduğunu söyledi.

7. Yerel Değişkenler ve Böl ve Fethet

Bir `f` fonksiyonu yazıp parantez içine `int a` dediğimizde bu değişken, çağrıda gönderilen argümanı parametre olarak almak için tanımlanır. Bloğun içinde de ayrıca değişken tanımlayabiliriz, örneğin bir `x` ya da bir karakter. Bunların hepsi fonksiyona özel, **yerel değişkenlerdir (local variables)**. Başka bir yerden doğrudan erişilemezler.

Fonksiyonların genel faydası, **böl ve fethet (divide and conquer)** yaklaşımına uymalarıdır. Bu yaklaşım yalnız programlamada değil, tarihte ve farklı alanlarda da karşımıza çıkar. Büyük bir problemi çözemiyorsak küçük parçalara böleriz, küçük parçaları çözerek genel problemi çözeriz.

Yazılımda fonksiyonlar tekrar tekrar kullanılır. Örneğin `scanf`'i yeniden derlemeyiz, derlenmiş hâlini çalıştırırız. Böylece kod tekrarından da kaçınılır.

> **Hocanın vurgusu:** Kod tekrarı, kod kusuru (code smell) dediğimiz, kişilerin en fazla yaptığı kod kusurlarından biridir.

8. Fonksiyonun Genel Biçimi ve return

Genel biçim şudur:

```
dönüş_tipi fonksiyon_adı(parametre_listesi){
    tanımlamalar ve ifadeler
}
```

- **Dönüş tipi:** `void` olabilir, o zaman hiçbir şey dönmez. Ya da `int`, işaretçi, karakter veya `book` gibi kendi yazdığımız bir tip olabilir.
- **Fonksiyon adı:** Değişken adlandırma kurallarına benzer kurallara uyar. Ama yapılacak bir eylemi ifade ettiği için ad da eylem bildirmelidir: diziyi sırala, şunu yap gibi. Hoca, değişken adlarının daha durağan, fonksiyon adlarının daha dinamik ifadeler olduğunu söyledi.
- **Parametre listesi:** Araya virgül konarak birden fazla parametre tanımlanır.
- **Gövde:** Normal bir program yazar gibi değişken tanımları ve ifadeler yazılır.

Fonksiyon çalıştıktan sonra içindeki değişkenlerin bazılarının saklandığını, bazılarının saklanmadığını da söyledi ve bu konuya sonra döneceğini belirtti.

`return` kullanımı dönüş tipine bağlıdır. Dönüş tipi `void` ise içeride boş bir `return;` yazabilirsiniz. Dönüş tipi `void` değilse `return ifade;` yazılır. Bu ifade tek bir sabit sayı, bir değişken, matematiksel bir işlem ya da mantıksal bir işlem olabilir.

Parametreler yalnız `int x`, `int y` gibi tekil değişkenler olmak zorunda değildir. Bir `int a` dizisi de olabilir. Dizi parametresi yazılırken boyut yazılmaz. Boyut `n` gibi başka bir parametreyle ya da farklı yollarla alınır. Ayrıntısı aşağıda dizilerde geliyor.

9. Fonksiyon Prototipi

9.1 square Örneği

Slayttaki ilk tam program, gönderilen bir sayının karesini alan `square` fonksiyonuydu. Programın başında şu satır vardı:

```
int square(int y);
```

Bu bir **fonksiyon bildirimidir (function declaration)**. Şu anlama gelir: Bu programda `square` diye bir fonksiyon yazdım, bir tane `int` parametre alıyor ve bir `int` döndürüyor. Kodun devamında `main` bitiyor ama program bitmiyor. Fonksiyonu `main`'den sonra ekleyecekseniz, `main`'den önce derleyiciye "böyle bir fonksiyon ileride gelecek, haberin olsun" demek istersiniz. Bu satır **fonksiyon prototipidir** ve fonksiyonun tanımına uygun yazılmak zorundadır.

Altyazıda anlatılan kod şöyledir:

```
#include <stdio.h>
int square(int);
```

```c
int main(){
    int x;
    for(x=1;x<=10;x++){
        printf("%d ",square(x));
    }
    printf("\n");
    return 0;
}
int square(int y){
    return y*y;
}
```

Akış şöyledir:

1. `x` 1'den başlar, 10'a kadar birer artar.
2. `printf` içindeki `square(x)` çağrısında `x`'in değeri olan 1, `y`'ye kopyalanır.
3. Fonksiyonda 1×1 hesaplanır ve tamsayı olarak döndürülür.
4. Dönen değer `%d` ile ilişkilendirilir ve ekrana yazılır.
5. Bu, $x = 1, 2, \ldots, 10$ için toplam 10 kez yapılır, yani 10 fonksiyon çağrısı olur.

Ders sonrasında derlenip çalıştırılan çıktı:

```
1 4 9 16 25 36 49 64 81 100
```

Hoca, fonksiyonun içinde ekstra bir değişkenin de kullanılabileceğini gösterdi: `int t = y * y; return t;` yazmakla `return y * y;` yazmak aynı sonucu verir. Yalnız fazladan bir değişken kullanılmış olur. Kafanız karışıyorsa ek değişkenler kullanıp işi yapmak ve en son hâlini döndürmek serbesttir.

9.2 Prototip Nasıl Çıkarılır?

Prototipi iki yerden çıkarabiliriz.

Çağrıya bakarak: Çağrıda `x` gönderiliyor ve `x` `int` ise fonksiyon bir `int` alır. Dönen değer `%d` ile yazdırılıyorsa `%d` tamsayıya karşılık geldiği için fonksiyon `int` döndürür. Hoca, fonksiyonun tanımına hiç bakmadan prototipi tahmin edebileceğini söyledi.

Başlık satırını kopyalayarak: En kolay yoldur. Kodu yazan siz olduğunuz için fonksiyonun başlık satırını kopyalar, `main`'in üstüne yapıştırır ve sonuna noktalı virgül koyarsınız. Prototipin ayrıca kod yazmaya başlarken belirlenmesi gerekmez.

Prototipte parametre adı yazmaya gerek yoktur, yalnız tipi söylemek yeterlidir. Hoca, slaytta adın görünmesini kitabın yazarının kodu doğrudan kopyalamış olmasına bağladı. Ad yazılırsa da program çalışır, üstte `x`, aşağıda `y` yazılsa da çalışır.

Soru: Prototipteki parametre adını fonksiyondakinden farklı yazabilir miyiz? **Cevap:** `x` ve `y`'yi farklı yazabilirsin ama gerek yok, yalnız `int` yazmak yeterlidir.

Soru: Veri tipleri farklı yazılabilir mi? **Cevap:** Veri tipleri farklı olamaz. Üst de `int`, alt da `int` olacak. Etiketler farklı olabilir. Veri tipi başka şey, etiket başka şeydir.

Soru: `int square();` diye boş parantezle yazsak derleyici tipi kendi çıkaramaz mı? **Cevap:** Boş yazarsan bu, hiçbir parametre almayacak demektir. Aşağıda parametre alacağını söylüyorsak prototipte de söylememiz gerekir. Boş bırakırsan derleme zamanı hatası alırsın.

> **Ders sonrası not:** Hoca boş parantezin "parametre almaz" demek olduğunu ve derleme hatası vereceğini söyledi. C23 öncesi C'de büyük olasılıkla doğrusu şudur: Boş parantez "parametreleri belirtilmemiş" demektir ve derleyici argümanları kontrol etmez. Ders sonrasında `int square();` bildirimi ve `square(5)` çağrısı `gcc -Wall -std=c89` ile uyarısız derlendi ve 25 yazdırdı. Aynı kod `gcc -std=c23` ile "too many arguments" hatası verdi, yani hocanın anlattığı davranış yeni standartta geçerlidir. Tipi prototipte yazmak her durumda doğru yoldur.

Soru: Fonksiyonun içeriğini yukarıda yazarsak ne olur? **Cevap:** O zaman prototip yazmaya gerek yoktur, çünkü `main`'den önce fonksiyonun tanımını yapmış oluruz.

Soru: İki fonksiyonda da `int x` kullansak bellekteki adresleri aynı mı olur? **Cevap:** Hayır. Fonksiyon içindeki x'ler, y'ler ve parametrelerin tamamı yerel değişkendir, yalnız o fonksiyona aittir. `main`'le ilişkileri yoktur. Haberleşme yalnız parametreler üzerinden olur: Gönderilen değer buraya belli bir mantıkla, birebir ya da adres üzerinden kopyalanır. Tanımlanan bütün değişkenler fonksiyon tamamlandığında tamamen ölür, bir anlamda serbest kalır.

Soru: C#'ta modülleri farklı bir sayfada yazıp çağırıyorduk. C'de de öyle mi? **Cevap:** O sınıf konusudur, nesneye yönelik programlamayla şu an işimiz yok. Karıştırmayın.

9.3 Üç Sayının En Büyüğü

İkinci slayt örneği, üç sayı alıp en büyüğünü bulan `maximum` fonksiyonuydu. Prototipte yine parametre adları yazmaya gerek yoktu. Ana programda üç sayı tanımlanır, kullanıcıdan alınır ve fonksiyon bir kez çağrılır.

```
#include <stdio.h>
int maximum(int, int, int);

int main(){
    int number1, number2, number3;
    printf("Uc tamsayi girin: ");
    scanf("%d%d%d",&number1,&number2,&number3);
    printf("En buyuk: %d\n",maximum(number1,number2,number3));
    return 0;
}
int maximum(int x, int y, int z){
    int max=x;
    if(y>max){
        max=y;
    }
    if(z>max){
        max=z;
    }
    return max;
}
```

Algoritma bir varsayımla başlar: x en büyük değer olsun. y bundan büyükse en büyük y'dir. z bundan büyükse en büyük z'dir. Sonunda `max`'ta hangisi varsa o döndürülür.

Hoca `main`'deki adların x, y, z olmasının da sorun çıkarmayacağını söyledi, çünkü `main`'in değişkenleri `main`'e, `maximum`'unkiler `maximum`'a aittir. Biri diğerinden tanınmaz.

Slaytta program üç kez çalıştırılmıştı. Bir çağrı olduğu hâlde üç sonucun görülmesi bundandır:

- 22, 85, 17 girildiğinde: `max` önce 22, sonra $85 > 22$ olduğu için 85. $17 > 85$ yanlış. Sonuç 85.
- 85, 22, 17 girildiğinde: `max` 85 kalır. Sonuç 85.
- Üçüncü çalıştırmada da 85 bulunur.

Prototipler bize üç şey söyler: parametrelerin bilgisi, dönüş tipi ve fonksiyonun adı. Bu, derleme aşamasında kolaylık sağlar. Fonksiyonun tanımı (dikkat, bildirimi değil) onu kullanan koddan sonra geliyorsa prototip yazmak zorundayız.

9.4 Gizli Dönüşüm ve Biçim Belirteçleri

Tipler uyuşmadığında gizli (implicit) bir veri dönüşümü olabilir. `maximum`'u 3, 4, 5 yerine 3.6 gibi bir değerle çağırdığımızı düşünelim. Bu bir `double`'dır ve `double`, `int`'ten büyüktür. Büyüğün küçüğe çevrilmesi hatalara yol açabilir. Değer `int`'e dönüştürülür (3.6 için 3, ara adım eklendi). Bu hatalar genellikle derleme zamanı hatası olmaz, çalışma zamanında ya da sonuç hatası olarak karşımıza çıkar.

Hoca, prototip fikrini aslında kullandığımızı da söyledi. `printf` ve `scanf`'te `%lf` yazınca bir `double` yazdırılacağını ya da alınacağını, `%c` yazınca bir karakterin işleneceğini bildiririz. Bu

belirteçlere uymak zorundayız, çünkü fonksiyonlar buna göre yazılmıştır. `printf`'in içine bakılsa, gelen parametrelere göre ekrana ne yazılacağını belirleyen kontroller görülür. Başlık (header) dosyalarıyla şu an uğraşmayacağız.

10. Değerle Çağırma ve Referansla Çağırma

Fonksiyon çağrılarında iki farklı mekanizma vardır: **değerle çağırma (call by value)** ve **referansla çağırma (call by reference)**. Referanstan kastımız değişkenin adresidir, yani işaretçidir (pointer). Hoca, gerçek anlamda bir call by reference'ın C'de bulunmadığını baştan belirtti. Biz şimdilik işlemlerimizin çoğunu değerle çağırma üzerinden yapacağız.

10.1 Değerle Çağırma

Bir `f` fonksiyonunu `f(3, x, a + b)` diye çağırdığımızı, tanımın da `void f(int x, int y, double z)` olduğunu düşünelim.

- 3 değeri ilk parametreye kopyalanır. Bellekte iki tane 3 olur.
- `x`'in değeri, örneğin 10, ikinci parametreye kopyalanır. Bellekte iki tane 10 olur.
- `a + b`'nin sonucu üçüncü parametreye kopyalanır.

Bu, birebir bir kopyalamadır. Hoca buna derin kopyalama (deep copy) ya da klon dedi. Bütün argümanlar parametrelere birebir kopyalanır. Değerlerden biri `main`'de, diğeri `f`'te durur. `f`'in içinde parametreyi artırmak, başka bir değer atamak `main`'deki değişkene yansımaz: Slaytın ifadesiyle, fonksiyondaki değişiklikler orijinali etkilemez. Bu da kazara yapılan değişikliklerin önüne geçer.

Hoca sordu: `printf`'e bir değer gönderdiğimizde `printf` bu değeri değiştirir mi? Değiştirmez, yalnız yazdırır. O hâlde burada bir değer geçişi vardır.

> **Ders sonrası not:** Altyazıda hoca iki yerde (00:53:02 ve 00:55:49) değişkenin değiştirilmesini istiyorsak call by value kullanılacağını söylüyor. Doğrusu tersidir: Büyük olasılıkla kastedilen, fonksiyonun orijinal değişkeni değiştirmesini **istemiyorsak** değerle çağırmaktır. Değiştirmesini istiyorsak adres göndeririz. Hocanın `printf` ve `scanf` örnekleri de bunu gösterir.

Değerle çağırmanın kullanımı kolaydır: Değişkenin adı yazılır, değeri doğrudan gönderilir, tipini de ona göre yazmak gerekir.

10.2 scanf ve Adres

Hoca bir `int n` tanımlayıp `scanf`'i `&` koymadan çağırdı ve programı çalıştırdı:

```
int n;
scanf("%d", n);
```

Program hata verdi. Hocanın anlatımına göre `scanf` belleğin bir hücresine yazmaya çalışır. `n`'ye 14 girildiğinde belleğin 14 numaralı adresine erişmeye çalışır, işletim sistemi de "14 numaralı adrese erişemezsin" diyerek programı durdurur.

> **Ders sonrası not:** Hoca, girilen değerin adres olarak kullanıldığını söyledi. Doğrusu büyük olasılıkla şudur: `scanf("%d", n)` çağrısında adres olarak `n`'nin **çağrı anındaki değeri** gider, klavyeden girilen sayı ise o adrese yazılmaya çalışılan değerdir. Ders sonrasında iki deneme yapıldı. `n`'ye başka bir değişkenin adresi konup 15 girildiğinde, 15'in o değişkene yazıldığı görüldü. `int n=14;` ile 15 girildiğinde ise program segmentation fault ile durdu. İlk değer verilmemiş `n`'nin içinde ise rastgele bir değer vardır, bu yüzden hata büyük olasılıkla buradan gelir. `gcc -Wall` bu satır için `%d`'nin `int *` beklediğini söyleyen uyarı verir.

Doğrusu `&` ile adresi göndermektir:

```
int n;
scanf("%d", &n);
```

`n` için bellekte bir yer ayrılır ve girilen değer `n`'nin bellekteki adresine yazılır. `scanf`'ten önce `n`'nin içinde belirli bir değer yoktur, klavyeden ne girilirse fonksiyon `n`'yi ona göre değiştirir ve

değişiklik çağrıldığı yere yansır. Sonrasında `n++` yazılırsa girilen değerin bir fazlası elde edilir. Hoca, burada call by reference olduğunu, ama bunun tam bir call by reference değil, "yalancı call by reference" olduğunu söyledi.

10.3 Referansla Çağırma

Referansla çağırmada değişkenin adresi gönderilir. Karşı tarafta bu adres, işaretçi konusunda görülecek farklı bir biçimle alınır. Adres gönderildiği için fonksiyonda yapılan değişiklik orijinale yansır. Slaytta bunun güvenilen fonksiyonlarda yapılması gerektiği yazılıydı: `printf`, `scanf` gibi fonksiyonlar ya da kendi yazdığınız fonksiyonlar. Hoca, işaretçilere yetişilirse bunun bir sonraki derste görüleceğini söyledi.

Bunun bir faydası daha vardır. Fonksiyonlardan `return` ile tek değer döndüğünü söylemiştik. `return a;` ya da `return 3;` yazılır, `return a, b;` diye bir şey yoktur. Ama fonksiyondan iki ya da üç değer almak da mümkündür. Bunu `return` ile değil, referansla çağırmayla yaparız: `g` fonksiyonunu `x`'in adresiyle, `y`'nin adresiyle çağırırız. Bunu `scanf`'te zaten görüyoruz:

```
scanf("%d %d", &a, &b);
```

Burada iki `%d` okunur, birincisi birinci değişkenin adresine, ikincisi ikinci değişkenin adresine yazılır. İki değerin de fonksiyon içinde değiştirilip çağrıldığı yerde kullanılabilmesi, referansla çağırma sayesindedir.

Hoca bu slaytlardaki rastgele sayı üretimi, depolama sınıfları (storage classes) ve kapsam (scope) konularını şimdilik geçti. Özyineleme (recursion) de ayrıca konuşulacak.

11. Dizilerin Özellikleri

Aradan sonra hoca, dizilerin fonksiyonlara nasıl aktarılacağına geçmeden önce dizilerin bazı özelliklerini hatırlattı.

Diziler bellekte ardışık saklanan elemanlardan oluşan bir veri yapısıdır. Bir dizinin adı vardır, örneğin `c`. Bir de elemanın pozisyon numarası vardır, örneğin `c[4]`. İlk eleman daima 0. pozisyonda, son eleman $n-1$. pozisyondadır.

Çoğu kişi için yeni olan bilgi şudur: Dizinin adı, 0. elemanının adresini verir. `int c[100];` yazıp daha sonra yalnız `c` yazdığınızda 0. elemanın adresini elde edersiniz.

> **Ders sonrası not:** Hoca 0. elemanın adresinin `c` değişkeninde saklandığını söyledi. Doğrusu büyük olasılıkla şudur: `c`, adres saklayan ayrı bir değişken değildir, ifadede ilk elemanın adresine dönüşür. Ders sonrasında 5 elemanlı `int` dizi için `sizeof(array)` 20 bayt çıktı, `array`, `&array[0]` ve `&array` aynı adresi gösterdi.

Dizi elemanları normal değişkenler gibi kullanılır. Slaytta `c[0]`'a 3 atanması ve `c[5-2]` ile `c[3]`'ün karşılaştırılması vardı. `x`'in içinde 3 varsa `c[x]` de aynı elemanı gösterir. Hepsi aynı yeri gösterdiği için karşılaştırmanın sonucu doğrudur (true).

11.1 İlk Değer Verme

Dizilere ilk değer verme kuralları şöyledir:

- 5 elemanlı bir diziye 5 değerin tamamı yazılırsa bu değerler kopyalanır.
- `int n[5] = {0};` yazılırsa bütün elemanlar 0 olur. En az bir değer yazmak yeter.
- `int n[5] = {1};` yazılırsa ilk eleman 1, geri kalanlar 0 olur.
- Boyut yazılmazsa (omit edilirse) yazılan eleman sayısı kadar yer açılır.

Slaytta 10 elemanlı bir dizinin başta 0 yapılıp `%7d` ve `%13d` ile tablo gibi yazdırılması vardı. `Element` başlığı 7, `Value` başlığı 13 karakter yer kapladığı için sütunlar düzgün hizalanıyordu.

Hoca birkaç örnekten daha hızlıca geçti. Bir diziye `j`'ye bağlı değerler atayıp 2, 4, 6, 8 diye giden bir sonuç elde etmek ve bir dizinin bütün elemanlarını `for` döngüsüyle toplamak basit algoritmalar olduğu için üzerinde durmadı.

Değerleri 1 ile 10 arasında olan bir dizideki her değerden kaç tane olduğunu 10 elemanlı bir sıklık dizisiyle saymak ise histogram oluşturmaktır. Bu sıklıklardan yola çıkıp her değer için

o sayıda yıldız basarak histogram çizdirme örneği (birinde 19, diğerinde 3 yıldız) ve zar atma örneği sonraya bırakıldı.

12. Karakter Dizileri

Karakter dizileri aslında bir string'dir, bir kelime ya da cümle saklar. Normal tamsayı dizilerinden farkı, sonuna otomatik olarak sonlandırma karakteri (termination character) `'\0'` eklenmesidir.

```
char string1[] = "first";
```

Bu dizi 5 değil 6 karakterlik yer kaplar, fazlalık `'\0'`'dan gelir. Eşdeğeri şudur:

```
char string1[] = {'f', 'i', 'r', 's', 't', '\0'};
```

Boyut yazılmadığı için bu da 6 karakterlik bir dizidir. Elemanlara tek tek erişilebilir ve indis yine 0'dan başlar: `string1[3]`, `s` karakteridir (ara adım eklendi).

`scanf` ile karakter dizisine okurken `&` koymaya gerek yoktur. Çünkü dizinin adı zaten ilk elemanın adresini söyler. Okunan her karakter bu adresten başlayarak diziye kopyalanır:

```
scanf("%s", string2);
```

`%s` yalnız boşluk (white space) görene kadarki karakterleri alır.

Karakter karakter ilk değer verirken `'\0'` elle eklenmezse sorun çıkar. Çift tırnaklı yazımda derleyici `'\0'`'ı otomatik koyar, tek tırnaklarla dizi oluştururken koymaz. Bu durumda yazdırırken bellekteki sonraki karakterler de dikkate alınır ve anlamsız karakterler görülebilir.

> **Ders sonrası not:** Altyazıda hoca bu durumda yazdırmanın "white space görene kadar" ilerlediğini söylüyor. `%s` ile yazdırma büyük olasılıkla boşlukta değil, `'\0'` görene kadar ilerler. Boşlukta durma `scanf`'in `%s` ile okumasına özgüdür. Hemen aşağıdaki "string literal" örneği de boşluğun yazdırmada durdurmadığını gösterir.

Slayttaki son string örneğinde iki dizi vardı: Biri klavyeden okunuyor, diğeri içinde boşluk bulunan `"string literal"` değeriyle başlatılıyordu. İkisi önce `%s` ile yazdırılıyor, sonra `'\0'` görene kadar `%c` ile karakter karakter yazdırılıyordu.

Klavyeden `Hello there` girildiğinde birinci dizi yalnız `Hello`'yu alır, çünkü okuma boşlukta durur. Ekrana da `Hello` yazılır. İkinci dizi `string literal` olarak bütünüyle yazılır, çünkü yazdırma boşluğu dikkate almaz, sonundaki `'\0'`'a bakar. Ders sonrasında `Hello there` girdisiyle yapılan denemede de `Hello` okundu.

Hoca `static` dizi kısmını şimdilik geçti.

13. Dizilerin Fonksiyonlara Aktarılması

13.1 Dizi ve Eleman Nasıl Gönderilir?

Bir diziyi fonksiyona argüman olarak göndermek, adını ve boyutunu göndermekle olur. 24 elemanlı bir `myArray` için çağrı şöyledir:

```
modifyArray(myArray, 24);
```

Gönderilen etiket, ilk elemanın adresini taşır. Burada yapılan mekanizma referansla çağırmadır. Fonksiyon dizinin gerçek yerini bilir, dolayısıyla elemanlarda yaptığı değişiklik orijinali de değiştirir.

Özetle `int` veya `float` bir değişken gönderirken değerle çağırma, `int`, `float` ya da karakter dizisi gönderirken referansla çağırma olur ve yalnız adı yazmak yeterlidir.

Dizinin tek bir elemanını gönderdiğimizde ise yine değerle çağırma olur. Çünkü `a[5]` de zaten bir `int`'tir, `int` dizisi `int`'lerden oluşur. `myArray[3]` yazıldığında 3 numaralı hücredeki değer kopyalanır ve fonksiyonda yapılan değişiklik yansımaz.

> **Ek bilgi (derste anlatılmadı):** Dizilerin "referansla" gittiği söylense de teknik olarak C'de her şey büyük olasılıkla değerle aktarılır. Dizinin adı ilk elemanın adresine dönüşür ve bu adresin bir kopyası gönderilir. Fonksiyon bu adres üzerinden orijinal

elemanlara eriştiği için değişiklikler orijinale yansır. Hocanın “gerçek call by reference C’de yoktur” sözü de bu duruma karşılık gelir.

13.2 Prototip ve Parametre Yazımı

Fonksiyon dizi alacaksa parametrede köşeli parantez yazılır, içine boyut yazılmaz. Etiketin ne olduğu önemli değildir. Boyut ayrı bir parametredir:

```
void modifyArray(int b[], int size);   /* tanım başlığı */
void modifyArray(int [], int);         /* prototip */
```

`size` genellikle `int` olmalıdır, çünkü eleman sayısı 3 ya da 5 olur, 3,5 olmaz.

> **Ek bilgi (derste anlatılmadı):** Boyutun ayrıca gönderilmesi gerekir, çünkü fonksiyona büyük olasılıkla yalnız ilk elemanın adresi ulaşır. Adresten dizinin kaç eleman içerdiği anlaşılamaz. Karakter dizilerinde bu sorun `'\0'` ile çözülür, tamsayı dizilerinde ise böyle bir son işareti yoktur.

13.3 Dizinin Adresi

Slayttaki ilk örnekte 5 elemanlı bir dizi için üç şey yazdırılıyordu: `array`, `&array[0]` ve `&array`. Üçü de dizinin ilk elemanının bellekteki adresini verir, çalıştırma sonucu da aynı adresi gösteriyordu. Bu üçünden hangisi gönderilirse gönderilsin, fonksiyon tarafında bir adresle karşılanması gerekir.

13.4 modifyArray ve modifyElement

> **Hocanın vurgusu:** Bu örnek çok çok önemlidir. Bu dersten tekrar edeceğiniz bir şey varsa en önemli örneklerden biridir. Gerekirse PDF’ten oturup tekrar tekrar yazın, hata yapın, hatanızı bulun, tekrar yazın. Anlayarak ilerlerseniz bu konuyu öğrenirsiniz.

```
#include <stdio.h>
#define SIZE 5
void modifyArray(int [], int);
void modifyElement(int);

int main(){
    int a[SIZE]={0,1,2,3,4};
    int i;
    for(i=0;i<SIZE;i++){
        printf("%3d",a[i]);
    }
    printf("\n");
    modifyArray(a,SIZE);
    for(i=0;i<SIZE;i++){
        printf("%3d",a[i]);
    }
    printf("\n");
    printf("a[3]=%d\n",a[3]);
    modifyElement(a[3]);
    printf("a[3]=%d\n",a[3]);
    return 0;
}
void modifyArray(int b[], int size){
    int j;
    for(j=0;j<size;j++){
        b[j]*=2;
    }
}
void modifyElement(int e){
    e*=2;
```

```
    printf("modifyElement icinde: %d\n",e);
}
```

Adım adım izlersek:

1. Başlangıçta dizi `0 1 2 3 4`'tür ve `main` içinde yazdırılır.
2. `modifyArray(a, SIZE)` çağrılır. Fonksiyon her elemanı 2 ile çarpar: $0 \times 2 = 0$, $1 \times 2 = 2$, $2 \times 2 = 4$, $3 \times 2 = 6$, $4 \times 2 = 8$.
3. Fonksiyonda `return` yoktur, çünkü `void`'dir. Ama dizi değer üzerinden değil, referans üzerinden geldiği için değişir. `main`'de `0 2 4 6 8` yazdırılır.
4. `a[3]`'ün değeri yazdırılır: 6.
5. `modifyElement(a[3])` çağrılır. İçeride $6 \times 2 = 12$ yapılır ve ekrana 12 yazılır. Ama bu 12 orada kalır.
6. `main`'e dönüldüğünde `a[3]` yine 6'dır.

Ders sonrasında derlenip çalıştırılan çıktı:

```
  0  1  2  3  4
  0  2  4  6  8
a[3]=6
modifyElement icinde: 12
a[3]=6
```

Yani dizide yapılan değişiklik yansıdı, elemanda yapılan değişiklik yansımadı.

14. Bubble Sort: Kâğıt Üzerinde

Hoca şimdi kendi örneğine geçti. Dizisini sıralı olmayan bir dizi kabul etti ve sıralamayı `main` yerine fonksiyonda yapacağını söyledi.

> **Hocanın vurgusu:** Fonksiyonlardan dizi dönmez, bunu bir kere bileceğiz. `return` ile dizi döndüremezsiniz. Başka dillerde bu vardır, C'de yoktur.

Bu yüzden sıralama fonksiyonu `void` olacak ve parametre olarak dizi ile boyutu alacaktı. Aynı yerde planladığı fonksiyonları da yazdı: sıralama, yazdırma ve bir elemanın kaçıncı hücrede olduğunu bulan arama fonksiyonları. Arama fonksiyonları hücre bilgisini `return` ile verecek, eleman yoksa -1 döndürecekti. İkili arama ve interpolasyon araması sıralı dizi gerektirdiği için önce sıralama yapılacaktı.

Bubble sort küçükten büyüğe sıralarken yan yana iki elemana bakar. Sıra bozuksa ikisini yer değiştirir (swap). Bir tur boyunca bu işi dizinin başından sonuna yapar. Hocanın kâğıt üzerindeki örneği 6, 8, 7, 5, 3 dizisiydi.

1. tur:

1. 6 ve 8: Kuralı bozan bir şey yok.
2. 8 ve 7: Bozuk, yer değiştir. Dizi 6, 7, 8, 5, 3.
3. 8 ve 5: Bozuk, yer değiştir. Dizi 6, 7, 5, 8, 3.
4. 8 ve 3: Bozuk, yer değiştir. Dizi 6, 7, 5, 3, 8.

Birinci turun sonunda en büyük eleman kesin olarak dizinin en sonuna itelendi. Aynı şekilde ikinci tur en büyük ikinci elemanı yerine gönderecek.

2. tur:

1. 6 ve 7: Değişiklik yok.
2. 7 ve 5: Yer değiştir. Dizi 6, 5, 7, 3, 8.
3. 7 ve 3: Yer değiştir. Dizi 6, 5, 3, 7, 8.
4. 7 ve 8: Değişiklik yok.

3. tur:

1. 6 ve 5: Yer değiştir. Dizi 5, 6, 3, 7, 8.
2. 6 ve 3: Yer değiştir. Dizi 5, 3, 6, 7, 8.
3. Sonrasında değişiklik gerekmez.

4. tur:

1. 5 ve 3: Yer değiştir. Dizi 3, 5, 6, 7, 8.
2. 5 ve 6, 6 ve 7, 7 ve 8: Değişiklik yok.

5. tur: Bütün karşılaştırmalarda değişiklik gerekmez. Demek ki dizi sıralandı ve burada bitirilir.

Bu adımların hepsi ders sonrasında Python ile izlendi ve aynı ara diziler bulundu.

Soru: Her turda en büyük sayı sona gittiğine göre, ikinci turda son sayıyı, üçüncü turda son iki sayıyı karşılaştırmayabilir miyiz? **Cevap:** Ona şu an gerek yok. Şu an naif bir algoritma yazıyoruz, iyileştirme kısmına daha sonra bakacağız.

15. Bubble Sort ve Dizi Yazdırma Fonksiyonlarını Kodlamak

Turları saymak için `tur`, ikili karşılaştırmalar için `i` değişkeni tutulur. İç içe iki döngü kurulur. Dış döngü turları sayar, iç döngü `a[i]` ile `a[i+1]`'i karşılaştırır. Hoca, `n` eleman için en fazla `n` tur yapılacağını söyledi ve bir tur boyunca hiç değişiklik olmazsa algoritmadan çıkma iyileştirmesini de sonraya bıraktı. Yer değiştirme için geçici bir değişken kullanıldı. Ders ekranındaki kod şöyleydi:

```
void bubble_sirala(int a[], int size){
    int tur, i, t;
    for(tur=1;tur<size;tur++){
        for(i=0;i<size-1;i++){
            if(a[i]>a[i+1]){
                t=a[i];
                a[i]=a[i+1];
                a[i+1]=t;
            }
        }
    }
}
```

> **Ders sonrası not:** Hoca `n` tur dedi, kod ise `tur`'u 1'den `size - 1`'e kadar çalıştırır, yani $n-1$ tur yapar. Bu büyük olasılıkla yeterlidir: Elle izlenen örnekte de dizi 4. turda sıralanmıştı, 5. tur yalnız kontroldü. Ayrıca hoca `i < size-1` sınırını "son eleman zaten en büyük" diye açıkladı. Asıl gerekçe büyük olasılıkla şudur: `i` son indise gelirse `a[i+1]` dizinin dışına çıkar.

Fonksiyon bittiğinde orijinal dizi sıralanmış olur. Yazdırma işi için de hoca derste yazdığı döngüyü `dizi_yazdir` fonksiyonuna taşıdı. Yalnız burada kendi `i` değişkenini tanımlaması gerekti. Dikkat edelim: `bubble_sirala`'daki `a`, `dizi_yazdir`'daki `a` ve `main`'deki `a` hep `a` adını taşır, ama her biri kendi fonksiyonunun yerel adıdır.

```
void dizi_yazdir(int a[], int size){
    int i;
    for(i=0;i<size;i++){
        printf("%2d-",a[i]);
    }
    printf("\n");
}
```

Ekrandaki kodda `printf` satırının yanında `//call by value` açıklaması vardı, çünkü `printf`'e `a[i]`'nin değeri gider. Daha önce yazılan `altsatiragec` fonksiyonuna da artık gerek kalmadı. Çağrılar şöyle yazıldı: önce dizinin orijinali yazdırılır, sonra sıralanır, sonra son hâli yazdırılır.

```
dizi_yazdir(a,MAX);
bubble_sirala(a,MAX);
dizi_yazdir(a,MAX);
```

Prototipler için kodu yazan biz olduğumuz için en kolay yol kullanıldı: başlık satırı kopyalandı, sonuna noktalı virgül kondu, parametre adları (etiketler) silindi.

```
void bubble_sirala(int [], int);
void dizi_yazdir(int [], int);
```

Henüz içi boş olan arama fonksiyonları, derlemede hata alınmasın diye `return 0;` ile bekletildi.

Bu sırada hoca `void` fonksiyonda da `return` kullanılabildiğini tekrar söyledi. Yalnız yanına bir değer yazılamaz. C buna izin verir, çünkü fonksiyonun ortasında değer döndürmeden çıkmak isteyebiliriz.

> **Ders sonrası not:** Hoca Java ve C#'ta `void` bir metotta doğrudan `return` kullanılamadığını söyledi. Bu iki dilde de `return;` büyük olasılıkla geçerlidir, değer yazılması yasaktır. Bu derste C için söylenenler doğrudur.

Soru: Yukarıda yaptığımız şey tanımlama mı? Bunu yapmazsak çalışır mı? **Cevap:** Bazı C derleyicilerinde çalışır, bazılarında çalışmaz. Onun için yapmamız gerekiyor. İkinci sınıfta bunları başlık (header) dosyalarına alacaksınız.

> **Ders sonrası not:** Hocanın "bazı derleyicilerde çalışmaz" sözü denemeyle de uyumludur. `swap`'ın prototipsiz hâli `gcc -Wall -std=c89` ile yalnız uyarı verdi, gcc 15'in varsayılan ayarıyla ise "implicit declaration" hatası verdi. Hatalı olmasa da büyük olasılıkla her ortamda prototip yazmak en güvenli yoldur.

16. swap Deneyi: Değer ile Referansın Farkı

Hoca, sıralama fonksiyonunun diziyi değiştirdiğini göreceğimizi söyledi. Bunu karşılaştırmak için bir de iki değişkeni yer değiştirmesi beklenen bir fonksiyon yazdı:

```
void swap(int a, int b){
    int t;
    t=a;
    a=b;
    b=t;
}
```

`main`'de iki değişken tanımlayıp `swap` çağrısından önce ve sonra yazdırdı:

```
int x=4, y=5;
/* ... */
printf("%d - %d\n", x,y);
swap(x,y);
printf("%d - %d\n", x,y);
```

Hoca bu sırada bir kuralı da hatırlattı: Değişkenler C'de daima işlemlere gelmeden tanımlanır.

> **Ders sonrası not:** Bu kural büyük olasılıkla C89 içindir. C99 ve sonrasında değişken kod içinde de tanımlanabilir. Ders sonrasında işlemden sonra tanımlanan bir değişken `gcc -std=c99 -pedantic` ile uyarısız, `gcc -std=c89 -pedantic` ile uyarıyla derlendi.

Program çalıştırıldığında görülenler şunlardı:

- Dizinin orijinali 20, 54, 7, ... diye gidiyordu. Sıralandıktan sonra 7, 8, 9, ... oldu ve değişiklik `main`'den görüldü.
- `swap`'tan önce 4 ve 5 yazıldı. `swap`'tan sonra da 4 ve 5 yazıldı. Yer değiştirme görünmedi.

Niye? Normal değişkenler değerleriyle kopyalanır, diziler ise referansla gider. Referansla gitmek orijinali değiştirmeyi sağlar. Hoca farkı çağrılar üzerinde şöyle gösterdi:

- `bubble_sirala(a, MAX)`: `a` referansla (pass by reference), `MAX` değerle (pass by value) gider.
- `swap(x, y)`: İkisi de değerle gider. Fonksiyonda değiştirilse de orijinal değişmez.
- `dizi_yazdir(a, MAX)`: `a` yine referansla gelir, ama fonksiyon dizide bir değişiklik yapmaz, yalnız yazdırır.

Soru: `swap`'ı tam anlamadım, ne işe yarıyor? **Cevap:** İki şeyin, yani `a` ile `b`'nin yerini değiştirir. Ama bu değişikliğin sonucunu `main`'de göremedik. `x`'in değeri `a`'ya kopyalandı. Bu yerel değişkenler kendi içinde yer değiştirdi ve bu hiçbir şekilde `main`'e yansımadı. Sebebi değerle çağırma mekanizmasıdır.

Hoca `swap`'ın prototipini yazmamıştı, derleyici bunun için uyarı da verdi. Ekranda şu mesajlar görünüyordu: `conflicting types for 'swap'` ve `previous implicit declaration of 'swap' was here`. Daha önce `altsatiragec` için de aynı uyarı çıkmıştı. Prototip eklenince bu uyarı kalkar.

Ekrandaki ilk çalıştırmada ise `printf` satırlarında henüz `\n` olmadığı için çıktı `4 - 54 - 5` diye bitişik görünüyordu.

17. Sıralı Arama

Hoca ikili aramaya geçmeden önce kolay olanı, **sıralı aramayı (sequential search)** yazdı. Bilinen bir algoritmadır. Bir de `key` parametresi gerekiyordu: Aranan eleman. Bu parametre arama fonksiyonlarının hepsine eklendiği için prototiplere de üçüncü bir `int` eklendi. Prototipi yazılmayan fonksiyon için derleyici yine uyarı verdi.

Hoca önce `i`'yi -1'den başlatmayı düşündü, sonra 0'dan başlatmanın da olacağını gördü. Fonksiyonun son hâli şudur:

```
int sirali_ara(int a[], int size, int key){
    int i=0;
    while(i<size && a[i]!=key){
        i++;
    }
    if(i==size)
        return -1;
    else
        return i;
}
```

`i` ya `size`'a gelene kadar ya da eleman bulunana kadar artar. Döngü bittiğinde `i == size` ise sonuna kadar gelinmiş ve bulunamamıştır, -1 döndürülür. Değilse `i` döndürülür. Bir fonksiyonda şarta bağlı iki `return` bulunmasının bir engeli yoktur.

Çağrılar, çift iş yapmamak için aynı anda iki örnekle yazıldı:

```
printf("Sirali arama=%d\n",sirali_ara(a,MAX,14));
printf("Sirali arama=%d\n",sirali_ara(a,MAX,88));
```

Sıralanmış dizi 7, 8, 9, 11, 13, 14, 18, 19, 20, 54'tür. 14'ün indisi 5'tir, çünkü sayım 0'dan başlar: 7 (0), 8 (1), 9 (2), 11 (3), 13 (4), 14 (5). Bu yüzden ilk çağrı 5 döndürür. 88 dizide olmadığı için ikinci çağrı -1 döndürür.

18. İkili Arama

18.1 Fikir

Hoca, Sayısal Analiz'de görülen bisection yöntemini hatırlattı. **İkili arama (binary search)** bu yöntemin dizideki hâlidir ve dizinin sıralı olmasını gerektirir. Elemanlara tek tek bakmak yerine ortadaki elemana bakılır. Aranan değer ortadakinden büyükse artık yalnız büyük değerlerin olduğu tarafa, küçükse küçük değerlerin olduğu tarafa bakılır.

Üç indis tutulur:

- `low`: Başta 0. elemanı gösterir.
- `high`: Başta `size - 1`'i, yani son elemanı gösterir.
- `mid`: `(low + high) / 2`.

Hocanın tahtaya çizdiği şekil şöyledir:

```
low             mid             high
 |               |               |
[ ... küçükler ...][ ... büyükler ... ]

aranan > a[mid]  ise  low  = mid + 1
```

```
aranan < a[mid]  ise  high = mid - 1
her iki durumda       mid  = (low + high) / 2
```

18.2 Kodlama ve Sorular

Döngünün şartı iki parçadır: `low`, `high`'tan küçük ya da eşit olduğu sürece ve `a[mid]`, aranan `key`'den farklı olduğu sürece devam edilir.

Hoca `if (key > a[mid])` durumunda ne yapılacağını sınıfa sordu.

Soru: `low = mid` mi olacak? **Cevap:** `mid + 1` olacak. Çünkü `mid`'e zaten baktık, elimizde.

Sonraki `else if` dalında hoca bir an `low = mid - 1` yazdı.

Soru: Orada `high` olmayacak mı? **Cevap:** Evet, `high = mid - 1` olacak.

Döngünün içinde `mid` güncellenir. Hoca sonra eşitlik durumunu düşünmeye başladı.

Soru: Eşit olduğunda zaten döngüye girmeyecek miyiz? **Cevap:** Doğru. Döngü şartı `a[mid] != key` dediği için içeride eşitlik olamaz. Bu yüzden `else if` yerine doğrudan `else` yazılabilir.

Döngü bittikten sonra iki olasılık vardır. `high < low` ise eleman bulunamamıştır, -1 döndürülür. Değilse `mid` bulunan yerdir ve döndürülür. `low` ile `high` eşitken döngü bir kez daha döner, bu yüzden bulunamama şartı eşitlik değil `high < low`'dur. Hoca bunu "`mid` 10. elemandı, `high` 9, `low` 11 oldu" gibi bir örnekle anlattı.

> **Ders sonrası not:** Sayılar anlatım içindir. `low` ve `high` 10'da eşitken bir adım sonra büyük olasılıkla ya `low` 11, `high` 10 ya da `low` 10, `high` 9 olur. Yani `low`, `high`'ın tam bir fazlası olur. Aşağıdaki 88 örneğinde de son durum `low` = 10, `high` = 9'dur. Şartın sonucu değişmez.

Fonksiyonun son hâli, sayaç dahil olarak aşağıdaki tam programdadır.

18.3 14'ü Aramak

Sıralı dizi ve indisleri şöyledir:

İndis	0	1	2	3	4	5	6	7	8	9
Değer	7	8	9	11	13	14	18	19	20	54

Adım	`low`	`high`	`mid`	`a[mid]`	Karar
1	0	9	4	13	$14 > 13$, `low = 5`
2	5	9	7	19	$14 < 19$, `high = 6`
3	5	6	5	14	Bulundu, indis 5

18.4 88'i Aramak

Adım	`low`	`high`	`mid`	`a[mid]`	Karar
1	0	9	4	13	$88 > 13$, `low = 5`
2	5	9	7	19	$88 > 19$, `low = 8`
3	8	9	8	20	$88 > 20$, `low = 9`
4	9	9	9	54	$88 > 54$, `low = 10`
Son	10	9	9		`low > high`, döngü biter, -1 döner

> **Ders sonrası not:** Hoca izlemeyi ekrana bakarak sözlü yaptı. Altyazıda 14 için "13, 19, 12" ve 88 için "13, 18 muhtemelen, 20" geçiyor. Python ile izlendiğinde 14 için bakılan değerler 13, 19, 14; 88 için 13, 19, 20, 54'tür. Dolayısıyla "12" ve "18" büyük olasılıkla dil sürçmesi ya da altyazı hatasıdır. Sonuçlar (indis 5 ve -1) değişmez.

19. Sayaçlarla Karşılaştırma ve Dersin Tam Programı

Hoca iki algoritmanın kaç kez döndüğünü görmek için sayaç ekledi. Sıralı aramada ayrı bir sayaca gerek yoktur, sayaç zaten `i`'dir. Değeri başka bir yere döndürmek gerekmediği için fonksiyonun içinde yazdırıldı. İkili aramada `i` diye bir değişken kullanıldı ve her güncellemede artırıldı. Önce 0'dan başlatıldı, sonra hoca döngüye girer girmez bir bakış yapıldığını söyleyip 1'den başlattı.

Son çıktıda şunlar görüldü:

- 14 aranırken sıralı aramanın sayacı 5, ikili aramanın sayacı 3 oldu.
- 88 aranırken sıralı aramanın sayacı 10 oldu, çünkü 10 elemanın sonuna kadar gelindi. İkili aramanın sayacı 5 oldu ve -1 döndü.

  Ders sonrası not: Sayaçlar bakılan eleman sayısıyla tam aynı değildir. 14'ü ararken sıralı arama 0'dan 5'e kadar 6 elemana bakar, hoca da bunu fark edip "aslında altı defa" dedi ama sıralı sayaç 0'dan başladığı için ekranda 5 kaldı. 88'de ikili arama 4 elemana bakar (13, 19, 20, 54), sayaç 1'den başladığı için ekranda 5 görünür. Bu yüzden hocanın "5 defa dönüyor" sözü büyük olasılıkla sayacın değerini anlatır, döngü gövdesi 4 kez çalışır. Karşılaştırmanın sonucu değişmez: İkili arama daha az elemana bakar.

Hoca ikili aramanın karmaşıklık analizine ayrıca bakılacağını söyledi. Burada görülen bilgi şimdilik yeterlidir.

Programın tamamı, ders ekranındaki hâle göre aşağıdadır. Interpolasyon araması fonksiyonu yalnız `return 0;` ile beklediği için buraya alınmadı. Ekran karelerinde görünmeyen parçalar (`sirali_ara`'nın baş satırları ve `main`'in son `printf` çağrıları gibi) anlatımdan tamamlandı. Kod ders sonrasında `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi.

```c
#include <stdio.h>
#define MAX 10
void bubble_sirala(int [], int);
void dizi_yazdir(int [], int);
void swap(int, int);
int sirali_ara(int [], int, int);
int binary_ara(int [], int, int);

int main(){
    int a[MAX]={20,54,7,8,9,11,13,14,18,19};
    int x=4, y=5;
    dizi_yazdir(a,MAX);
    bubble_sirala(a,MAX);
    dizi_yazdir(a,MAX);
    printf("%d - %d\n", x,y);
    swap(x,y);
    printf("%d - %d\n", x,y);
    printf("Sirali arama=%d\n",sirali_ara(a,MAX,14));
    printf("Sirali arama=%d\n",sirali_ara(a,MAX,88));
    printf("Binary arama=%d\n",binary_ara(a,MAX,14));
    printf("Binary arama=%d\n",binary_ara(a,MAX,88));
    return 0;
}
void swap(int a, int b){
    int t;
    t=a;
    a=b;
    b=t;
}
void dizi_yazdir(int a[], int size){
    int i;
    for(i=0;i<size;i++){
        printf("%2d-",a[i]);
```

```
    }
    printf("\n");
}
void bubble_sirala(int a[], int size){
    int tur, i, t;
    for(tur=1;tur<size;tur++){
        for(i=0;i<size-1;i++){
            if(a[i]>a[i+1]){
                t=a[i];
                a[i]=a[i+1];
                a[i+1]=t;
            }
        }
    }
}
int sirali_ara(int a[], int size, int key){
    int i=0;
    while(i<size && a[i]!=key){
        i++;
    }
    printf("Counter for sequential=%d\n",i);
    if(i==size)
        return -1;
    else
        return i;
}
int binary_ara(int a[], int size, int key){
    int low=0, i=1;
    int high=size-1;
    int mid=(low+high)/2;
    while(low<=high && a[mid]!=key){
        i++;
        if(key>a[mid]){
            low=mid+1;
        }
        else{
            high=mid-1;
        }
        mid=(low+high)/2;
    }
    printf("Counter for binary=%d\n",i);
    if(high<low){
        return -1;
    }
    else
        return mid;
}
```

Çıktı:

```
20-54- 7- 8- 9-11-13-14-18-19-
 7- 8- 9-11-13-14-18-19-20-54-
4 - 5
4 - 5
Counter for sequential=5
Sirali arama=5
Counter for sequential=10
Sirali arama=-1
Counter for binary=3
Binary arama=5
Counter for binary=5
```

```
Binary arama=-1
```

Ders ekranındaki son çıktıda binary satırları da “Sirali arama” etiketiyle yazıyordu. Hoca bunu fark edip “binary arama” diye düzeltti. Ekranda ikinci `printf`'te `\n` olmadığı için `4 - 5` ile `Counter for sequential=5` bitişikti, burada satır sonu eklendi.

> **Ders sonrası not:** Ders ekranındaki `main`'de `return 0;` yoktu ve pencerede `return value 5`, sonraki çalıştırmada `return value 16` görünüyordu. Bu sayılar büyük olasılıkla son `printf`'in yazdığı karakter sayısının kalıntısıdır (`4 - 5` 5 karakter, `Sirali arama=-1` ve satır sonu 16 karakter). Burada `return 0;` eklendi.

Dersin Toparlaması

Bu derste fonksiyonların neden gerektiğini kod tekrarı üzerinden gördük. Fonksiyon tanımını, çağrısını, argüman ile parametre farkını, dört tanım durumunu ve prototipi öğrendik. Argümanların iki biçimde gittiğini gördük: Tekil değişkenler ve dizi elemanları değerle gider, fonksiyondaki değişiklik orijinale yansımaz. Diziler referansla gider, değişiklik orijinale yansır. Değişkenleri de referansla göndermenin yoluna işaretçileri görünce bakılacak, `scanf` zaten böyle çalışıyor. Uygulama olarak bubble sort, sıralı arama ve ikili arama fonksiyonlarla kodlandı.

Sonraki derslerde interpolasyon araması kodlanacak, selection sort ve insertion sort görülecek, bubble sort'un zaman ölçümü ve iyileştirmeleri incelenecek. `swap` denemesinde karşılaşılan sorun da işaretçilerle çözülecek: Tek bir `swap` fonksiyonu, bütün sıralama algoritmalarının yer değiştirme işini yapacak. Hoca bundan sonra algoritmaları daha çok akış diyagramı yerine C kodu ve fonksiyonlarla göstereceğini söyledi. Fonksiyonlar anlatılmadan sıralama algoritmaları uzunca yazılsa, koca bir `main` içinde her şey birbirine karışırdı. Bu nedenle hem modüler program yazmak gösterilecek hem de algoritmik taraf ilerletilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.